

Software Synthesis for Embedded Processors

Raihan Uddin

Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
raihan.uddin@stud.hshl.de

Abstract—Optimized low-level software code, synthesized from high-level behavioral specifications, ensures efficient and accurate program execution on hardware by offsetting resource limitations. As hardware and software are parallelly developed to fulfill system-level goals, software synthesis can pave the path for early planning of the significant challenge in embedded field- the hardware constraint; thus, can improve upon performance, cost-efficiency and flexibility across both domains. C/C++/SystemC are the most common languages used for code automation. The targeted back-end hardware like MCUs, DSPs, FPGAs and AISPs are significant beneficiaries of the synthesis procedures. Wherever design parameters like power consumption, real-time deadlines, memory allocation are of critical concerns, hardware software seamless alignment is highly anticipated. The goal of the seminar paper is to explore some of the core aspects of software synthesis like scheduling tasks to processors. It will provide a comprehensive understanding of efficient mapping of software onto embedded processors while adhering to system-level critical constraints including timing, resource limitations and power consumption.

I. INTRODUCTION

According to [1], software synthesis is a process where a customized code is automatically generated from system-level specification. Today, there is always a push for market growth in the segment of Silicon industry. The demand for high performance, low power consumption, and cost-effective solutions is increasing. In quest of embedding many-processor systems on a single chip (MPSoCs), system complexities are growing at an almost exponential rate [2]. The demand for sophisticated high-performance, low-powered processors to meet the system complexities is unprecedented for today's applications. However, the traditional design processes, "in which systems are designed directly at the low hardware or software levels" [2], are fast becoming obsolete and infeasible to bridge the gap "generated by the disparity between the rapid paces at which design complexity has increased in comparison to that of design productivity" [2]. In this context, the software-synthesis approach offers a viable design solution.

Most major semiconductor roadmaps suggest a raise in the level of abstraction in the design process as an acceptable solution for closing the productivity gap [2]. Higher abstraction levels is more aligned with the system requirements and specifications. Software synthesis can bridge the semantic gap between higher abstraction levels and low-level implementation [2]. As stated in [3], software must be developed specifically for the given cores, platform, applications and mapping for Hardware-dependent-Software (HdS). Also de-

scribed by [3] that the manual HdS development can become time consuming not only due to the code size, but also due to the complex interaction processes in concurrent applications mapped to multi-core platforms. The software synthesis approach enables the automatic generation of code from high-level specifications, thus reducing the complexity and time required for manual coding. More specifically, this is a design methodology that allows designers to apply "design-automation techniques for modeling, simulation, synthesis, and verification to the system-design process" [2].

II. Y-CHART

Y-Chart, developed in 1983, also known as the Gajski-Y-Chart, is a design methodology that provides a framework for the design of complex systems, particularly in the context of hardware and software co-design. According to [2], the Y-Chart approach is characterized by its three main components or axes: behavior (functionality/specification), design structure (netlist/block diagram), and physical design (layout/board design). These components are represented in a Y-shaped diagram, where each arm of the Y represents a different aspect of the design process. The Y-Chart in figure 1 comprises

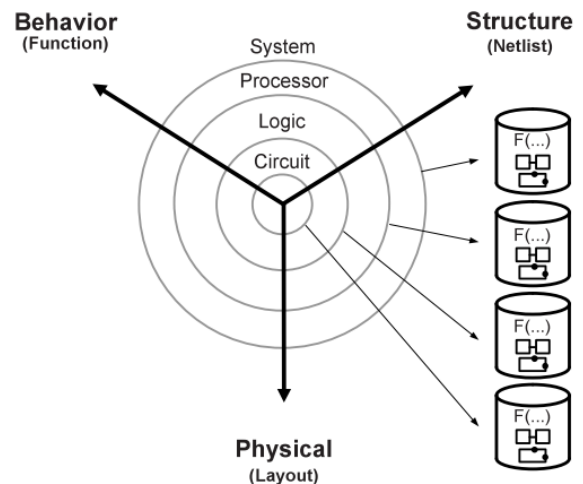


Fig. 1. Y-Chart approach [2].

three axes, each representing a different aspect of every design process [2]. The axes are as follows:

- **Behavior:** "What" the system should perform

- **Structure:** “How” the system is organized
- **Layout:** “Where” the system is implemented

The concentrated circles around the origin of the Y-Chart represent design on different abstraction levels and the name of the each abstraction level originates from the types of the components generated on that abstraction level [2]. The four levels of abstraction are as following:

- **Circuit level:** The components generated in this level are basic building blocks of the circuit i.e., N-type or P-type transistors.
- **Logic level:** This level utilizes logic gates and flip-flops to generate register-transfer components e.g., registers, register files and also functional units such as ALUs and multipliers [2].
- **Processor level:** At this level, standard and custom processors along with special-hardware components e.g., memory controllers, arbiters, bridges, routers, and various interface components are generated [2].
- **System level:** At this level, embedded systems are designed using processors, memories, buses and other processor components [2], generated on the lower levels.

The process of implementing a given behavior into a virtual structure on each abstraction level is known as synthesis. Each abstraction level should also contain a database of components that are comprising the structure. After synthesizing at one level, one can move to next lower abstraction level [2] and synthesize the structure further. However, if there is predefined library of components available with their structures and physical dimensions, one can proceed with the physical designing that consists of floorplanning, placement and routing on the chip or PC board [2]. Because of the modern day advancement in available software tools, it is possible to start designing or synthesizing directly from the system and processor levels without considering the lower levels. As stated in [2], “every system component except standard processors and memories is synthesized to the logic level, before the physical design is performed on the logic level”. This means that all custom components (e.g., controllers, accelerators etc.) except few are readily available to synthesize to the next lower level of abstraction that is logic level in the next step. These exceptional components- processors and memories are reused directly as IPs. In this regard, it is mentioned in [2] that “some components on the processor-and-system levels may be obtained as IPs and not synthesized. Therefore, their structure and physical design are known, at least partially, on the level higher than logic level. In that case, the physical design then may contain components of different sizes and from different levels of abstraction”.

III. ABSTRACTION AT PROCESSOR LEVELS

A. Behavior Modelling at Processor Levels

In behavioral modelling at processor level, the computational units— processors are defined and subsequently designed. According to [2], these processing elements (PEs)

can compute either specific functions (dedicated/custom components) for specific usage or generic computation (general/standard PE) of any functions.

Instead of classical approach of defining functionality with mathematical expressions and formulas, modern days graphical user interfaces have provided capability to replace them with flow chart specifications. One of such approach is Finite State Machine (FSM) modelling — a set of states and transitions. The transitions take place from state to state upon meeting the required value of some input variables [2]. The FSM also generates values for output variables during each transition of states. It is also noteworthy to mention here, as stated in [2] that, “the original FSM model uses binary variables for inputs and outputs”. FSMD is an iteration of FSM with variable values extending from integer to floating point along with binary. A set of arithmetic expressions or programming statements [2], executing in the background, will compute the variable values in extensive ranges during each state transition. However, the FSMD model is incapable of representing expressions of any programming language control statements. For that, there is also another model known as Control-Data Flow Graph (CDFG). This is considered to be a FSMD with superstates that require multiple clock cycles to execute [2]. Besides, it is also possible to define functionalities of these PEs with algorithm implementation in some programming languages (Instruction Sets).

B. Structure Modelling at Processor Levels

Once the behavior of the processing units are defined, they can be translated into virtual hardware components using registers, ALUs, Demultiplexers, Adders etc. These model contains a controller and a datapath where the datapath “consists of a set of storage elements (such as registers, register files, and memories), a set of functional units (such as ALUs, multipliers, shifters, and other custom functional units), and a set of buses” [2]. Based on the behavior of the processor, the type and number of components are chosen. For the internal connections among these components, buses, a network-on-chip (NOC) or crossbars can be used. Each component may require multiple clock cycles for execution and a control logic can be used for that purpose. To ensure correct timing and sequencing of data, each component can be pipelined—breaking complex blocks into smaller units along with the entire datapath [2].

According to [2], a controller “defines the state of the processor clock cycle per clock cycle and issues the control signals for the datapath accordingly”. However, the structure and implementation of the controller depends on— whether the processor, in question, is of standard type (like: Xeon, ARM, DSP), custom-designed or Intellectual Property (IP) function-contained processor for targeted platform [2]. Therefore, it can be said, in summary, that this structure level is mapping the behavioral descriptions, defined in the previous level, to RTL components with clock-cycles using pipeline, I/O latches or registers, and resource allocation mechanisms.

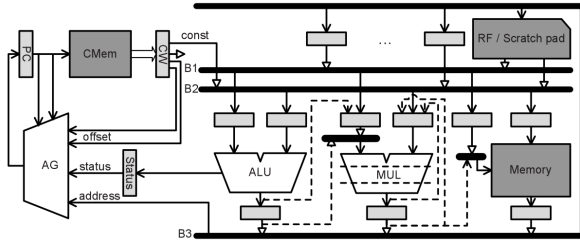


Fig. 2. Exemplary Structural Model of a Processor [2].

C. Synthesis at Processor Leveles

There is a difference in the process of synthesis among different processors— standard, custom or custom IPs. For standard processors, engineers design each part of the processor by hand- writing HDL code (VHDL/Verilog) for each unit (ALUs, multiplexer, decoders, cache memories, pipelines), tuning performance and power consumption variables and sometimes even hand-placing logic gates in critical component blocks. On the other hand, a custom processor or a custom IP design or synthesis begins with the C code of the pre-designed algorithm which is then converted to the corresponding CDFG or FSM model before synthesis [2]. After synthesis, it, then, turns out to be a custom entity with inter-connected various components, determined by the given behavioral model [2]— known as a processor. This virtual sketching of processor is

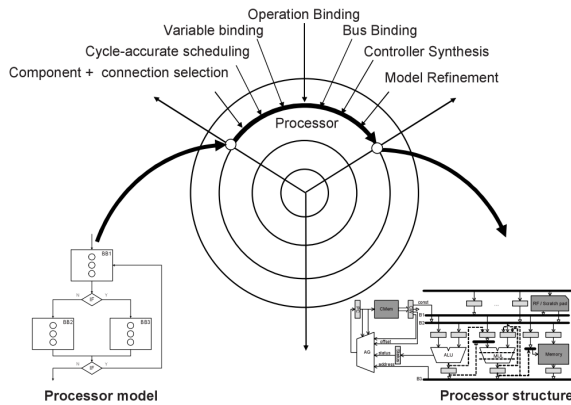


Fig. 3. Processor Synthesis [2].

known as high-level or register-transfer synthesis. According to [2], it consists of five individual tasks, which are as follows:

- Allocation of components and connections:** From existing hardware library, it is possible to start assembling the virtual processor. The components are readily available building blocks (like lego) which are verified and characterized with key metrics like— latency, area cost, power draw, delays, I/O port counts etc. For each dictated operation in the processor's behavior, a matching hardware block must have to be used. After binding (assigning operations to specific components) and scheduling (clock cycles of each operation) are specified, it is possible to

move to next process— putting connectivity among the components with wires, buses or a NoC. This helps in estimating all the delays [2] relating to the connectivity floorplan.

- Cycle-accurate scheduling:** Any operation that is supposed to be performed by the processor, defined in the behavioral model— must be scheduled into clock cycle [2]. Depending on the datapath, reading and writing variable values from and to the storage components may require multiple cycles. A generalized workflow can be as follows: [read operands from memory] → [execute operation] → [store result in memory]. A generalized scenario can be considered where we need to compute the multiplication of two operands and store it in 'd'. The cycles of the overall process can be visualized as follows: Cycle 1: Fetch b, c
Cycle 2: Execute $a = b + c$
Cycle 3: Store a, Fetch e
Cycle 4: Execute $d = a * e$ (starts)
Cycle 5-6: Multiplier pipeline continues
Cycle 7: Store d

It is evident that each operation, like the normal one above, "by itself may take several clock cycles in a pipelined datapath" [2].

- Binding of variables, operations and transfers:** Each variable must be contained in a storage unit. Then also, any outlined operation in the behavioral model must be assigned to corresponding functional units (hardware blocks). However, if there exist several units serving that capability, "the binding algorithm must optimize the selection" [2].
- Synthesis of controller:** The controller is a hardware block that sends control signals to instruct the processor, for example— when to fetch data, when to execute and operation, where to store results etc. There can be different types of controller can be used for processor synthesis— for example, programmable controller, ROM based for fixed functional IPs, or even pure logic-based controller.
- Model refinement:** It is the ultimate structural output as working processor in the virtual environment. As the synthesis gradually is moving towards implementation in hardware level, there are more and more details of the processor are added:

- 'What each operation is',
- 'Where the data are stored',
- 'Which functional unit (hardware block) is responsible for which operation'
- 'How data moves through busses or any connection'

There is always room to follow different styles to generate a new processor. According to [2], the execution of the statement $a = b + c$ can follow steps like follows:

a) No Binding – Behavioral Level

$$a = b + c;$$

It is only behavioral: only what should happen, not how.

b) Storage Binding

$RF(1) = RF(3) + RF(4);$

Binding variables to physical registers:

- a is stored in register file location $RF(1)$
- b and c are in $RF(3)$ and $RF(4)$

c) Storage + Functional Unit Binding

$RF(1) = ALU1(+, RF(3), RF(4));$

- The operation + is performed by ALU1
- Input operands are from $RF(3)$, $RF(4)$
- Output goes to $RF(1)$

d) Storage + Functional + Connectivity Binding

$Bus1 = RF(3);$

$Bus2 = RF(4);$

$Bus3 = ALU1(+, Bus1, Bus2);$

$RF(1) = Bus3;$

- Each data transfer is shown: from register $\rightarrow$ bus $\rightarrow$ ALU $\rightarrow$ bus $\rightarrow$ register

IV. IMPORTANCE OF SOFTWARE SYNTHESIS AT PROCESSOR LEVEL

Software synthesis produces binary codes for programmable processing elements [2]. As shown in Figure 4, a system model of a processor describes the components and the connectivity, whereas the bottom portion of the design flow takes the system model as an input and generates an implementation of each component [2]. Software development for embedded

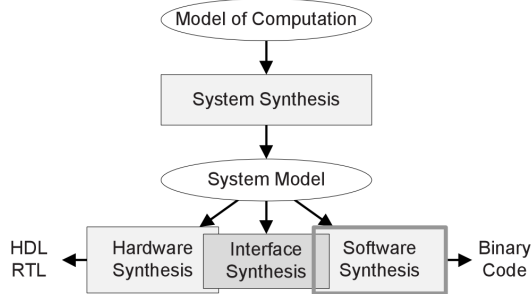


Fig. 4. Software Synthesis [2].

system covers most of the expenses of producing modern multi-processor systems in compared to hardware development. As there are more capabilities of any processor these days, they need firmware, drivers, OS code among others to work properly. Hence, lies the development of multi-faceted software programs to embed in one processor. However, hand-written codes, like conventional practice, do take lot of time for implementation. Hence, engineers, these days, prefer to implement features of processors in software tools because of their flexibility, reusability and for being easily updatable.

V. TARGETED LANGUAGES FOR EMBEDDED PROCESSOR

There are vast range of languages suitable to program embedded systems, like— assembly, C, C++, and Java. However,

there are differences in many aspects— “for example in the abstraction level at which the program is written and with that the granularity of control over the processor” [2]. Among them, assembly language represent the lowest level of abstraction in software development. That is why it is mentioned by [2] that the assembly is “basically a symbolic representation of the processor’s machine code and only minimally abstracts the processors complexity”. It gives control over the processor’s internals— such as controlling the registers and instruction-level operations. That is why, it is also valuable for writing performance-critical or low-level routines. However, assembly is very much dependent on the processor’s Instruction Set Architecture (ISA), meaning that each processor family (e.g., ARM, x86, RISC-V) differs in its own unique assembly language.

VI. IMPORTANCE OF RTOS

The languages, those are mentioned before, are unable to provide support for concurrency— which is integral part of embedded systems [2]. There lies the importance of RTOS existence in any embedded system. According to [2], embedded applications are often executed on top of an RTOS which allows concurrent task execution. It has, to some extent, similarities with a general purpose operating system. It is a software layer sitting above the bare processor and manages “concurrent execution of applications and provides various services for communication and synchronization” [2].

VII. SOFTWARE SYNTHESIS FLOW

In the figure 5, it is shown that the software synthesis takes a high-level model of a system and automatically generates the low-level embedded code, drivers, and binaries needed to run on a processor. It is, thus, eliminating the need for hard-coding, shortening the development time, reducing error-susceptibility, and allowing testing even before the real hardware is built — by using virtual platforms with instruction set simulators.

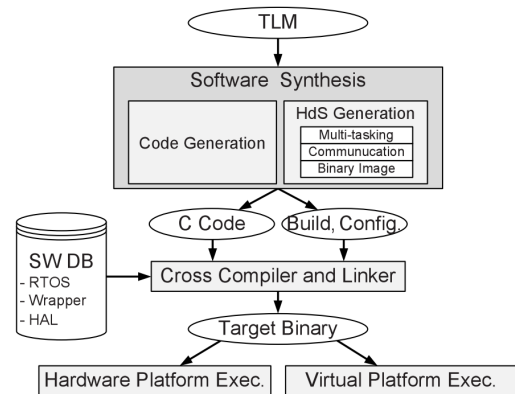


Fig. 5. Software Synthesis Flow [2].

VIII. TLM

TLM (Transaction-Level Modeling) is a way to simulate and design systems by describing high-level data transfers between components through busses. However, it utilizes different semantics instead of focusing on low-level signals or wires.

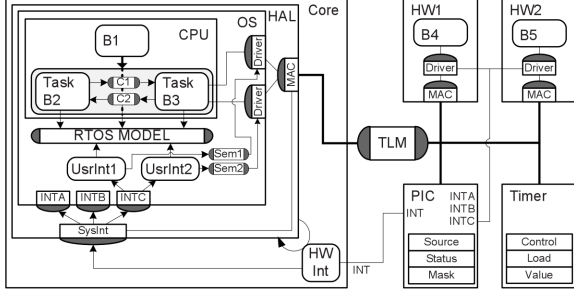


Fig. 6. TLM [2].

IX. TARGET ARCHITECTURE

Software synthesis typically targets a multi-core embedded platform instead of single processor. These platforms have several general-purpose processors and hardware accelerators, each with its own local memory, programmable interrupt controller (PIC), and timer for time-based tasks.

Processors are connected to the system through a processor bus, enabling communication with local components, memories etc.

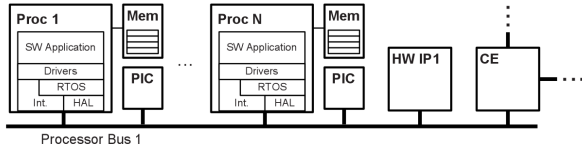


Fig. 7. Generic Target Architecture [2].

X. CODE GENERATION

Code generation in software synthesis produces sequential code, typically in C for each task that runs on a programmable processor. An exemplary code generation rule is as follows:

XI. SUMMARY

Overall, software synthesis narrows the gap between high-level system design and low-level executable code, offering faster development cycles, early validation, hardware-software consistency, and scalability across different architectures. It promises to perform more complex architectural design in future with more computing capacity.

```

1 SC_MODULE(B1){
2   int A;
3   sc_port<iChannel> myCh;
4   SC_CTOR(B1){}
5   void main(void) {
6     A = 1;
7     myCh->chCall(A*2);
8   }
9 };
10
11 SC_MODULE(TaskB2){
12   CH1 ch11, ch12;
13   B1 b11, b12;
14   SC_CTOR(TaskB2):
15     ch11("ch11"), ch12("ch12"),
16     b11("b11"), b12("b12") {
17     b11.myCh(ch11); // connect ch11
18     b12.myCh(ch12); // connect ch12
19   }
20   void main(void) {
21     b11.main();
22     b12.main();
23   }
24 };

```

Fig. 8. SystemC task specification [2].

REFERENCES

- [1] E. Tavares, B. Silva, P. Maciel, and P. Dallegrave, "Software synthesis for hard real-time embedded systems with energy constraints." [Online]. Available: <https://ieeexplore.ieee.org/document/4685735>
- [2] D. D. Gajski, S. Abdi, A. Gerstlauer, and G. Schirner, *Embedded System Design: Modeling, Synthesis and Verification*. Springer Science+Business Media, 2009.
- [3] S. Abdi, G. Schirner, H. Cho, Y. Hwang, and L. Yu, "Hardware-dependent software synthesis for many-core embedded systems." [Online]. Available: <https://ieeexplore.ieee.org/document/4796498>